

Modules & Packages

Organizing & Reusing Code in Python (OOP)

CMPE-103 • Object-Oriented Programming

Godofredo Avena
June 2026

Introduction

- In OOP, organizing and managing code effectively is crucial for building scalable, maintainable applications.
- Python provides this through modules — reusable pieces of code that encapsulate functions, classes, and variables.
- Modules align with core OOP principles:
 - Abstraction
 - Encapsulation
 - Modularity
- They let you break complex problems into manageable components — easier to collaborate on and maintain over time.

Why modules?

- ✓ Readability
- ✓ Reusability
- ✓ Maintainability
- ✓ Collaboration
- ✓ Scalability

Learning Objectives

- What modules in Python are
- Python import module
- How to create & import a module
- Importing a class in Python
- Using from to import a module
- Importing everything from a module
- Using import *
- The dir() function
- Packages
- Python module search path
- Using a module alias in imports
- Absolute & relative imports
- Using absolute imports
- Using relative imports

What Is a Module?

- A module is simply a file containing Python code.
- That code can be variables, functions, or class definitions.
- The filename becomes the module name — e.g. Python99.py
→ module Python99.
- Modules let you split code across files instead of one giant file.

A module = one .py file

```
# Python99.py

name = "Python99"

def greet():
    return "Hi!"

class Car:
    ...
```

The Python import Keyword

- A file is treated as a module in Python.
- To use it, import it with the import keyword.
- Functions / variables inside the file can then be used in another file.
- Same idea exists in other languages: TypeScript, JavaScript, Java, Ruby...

```
import test

# call with module_name.func
print(test.display_message())
```

Create & Import a Module — Step by Step

- Folder: modtest/ → test.py + display.py
- 1. Create test.py
- 2. Define a function display_message() inside it
- 3. Create display.py
- 4. In display.py write: import test
- 5. Call it via module_name.function_name
- 6. Run display.py → output appears

```
# test.py
def display_message():
    return "Welcome to Python99 Modules!"
```

```
# display.py
import test
print(test.display_message())
```

Output

```
Welcome to Python99 Modules!
```

Importing a Class

Define a class in its own file

```
# Car.py

class Car:
    def __init__(self, brand, model, year):
        self.brand_name = brand
        self.model = model
        self.manu_year = year

    def car_details(self):
        print("Car brand is", self.brand_name)
        print("Car model is", self.model)
        print("Year is", self.manu_year)
```

Use it as a module

```
# display.py

import Car

car = Car.Car("BMW", "Z5", 2020)
print(car.brand_name)
print(car.car_details())
```

Output

```
BMW
Car brand is  BMW
Car model is  Z5
Year is  2020
```

Using from to Import

- Import only the parts you need — specific functions or variables.
- Use the from keyword instead of importing the whole module.
- Imported names are accessed directly (no module prefix).
- Names you did NOT import raise a NameError if used.

Syntax

```
from module import name1, name2, ...
```

Example

```
from test import display_message  
print(display_message())  
# Welcome to Python99 Modules!
```

Not imported → error

```
print(display_message1())  
# NameError: 'display_message1'  
#   is not defined
```


Importing Everything from a Module

Two ways to access all names in a module — with or without a prefix.

a) `import module` — prefix required

```
import test

print(test.display_message())
print(test.my_name)
print(test.my_address)
```

b) `import *` — direct access

```
from test import *

print(display_message())
print(my_name)
print(my_address)
```

Both produce

```
Welcome to Python99 Modules! | All about Python! | Python99 | Mumbai
```

The dir() Function

- Built-in function that lists an object's properties & methods.
- On a module, it returns the variables, functions and classes inside it.
- Also works on built-in modules (e.g. json) and any object.

Usage

```
import Car
print(dir(Car))

import json
print(dir(json))
```

dir(json) → sample output

```
['JSONDecoder', 'JSONEncoder',
 'dump', 'dumps', 'load', 'loads',
 'decoder', 'encoder', 'scanner', ...]
```

Packages

- A package is a directory containing modules.
- Add an `__init__.py` file to make Python treat the directory as a package.
- Packages can also contain sub-packages (sub-folders).

Importing from a package

```
import mypackage.module1 as mod1
mod1.mod1_func1()
# Welcome to Module1 function1
```



mypackage/

```
└─ __init__.py
└─ module1.py   → mod1_func1/2/3()
└─ module2.py   → mod2_func1/2/3()
└─ module3.py   → mod3_func1/2/3()
```

Python Module Search Path

- When Python hits an import, it locates the module by searching, in order:
 - 1. The current directory
 - 2. The built-in module list
 - 3. The directories listed in `sys.path`
- Inspect the search path with the `sys` module — you can also modify it.

View `sys.path`

```
import sys
print(sys.path)

# ['.../task2',
#  '.../python37.zip',
#  '.../DLLs', ...]
```

Using a Module Alias

- Give a module a shorter name with the as keyword.
- Refer to its functions and variables through the alias.
- Common for long module names (e.g. import numpy as np).

Syntax

```
import <filename> as <alias>
```

Example

```
import test as t

print(t.display_message())
print(t.my_name)
print(t.my_address)
```



LESSON 2

Types of Imports in Python

Absolute vs. Relative Imports

Example Project Structure

```
myproject/
├── package1/
│   ├── module1.py → myfunc1()
│   └── module2.py → myfunc2()
├── package2/
│   ├── myclass.py
│   ├── subpkg/ → module3.py → myfunc3()
│   └── module4.py → myfunc4()
```

- Root folder: myproject/
- Two sub-folders: package1, package2
- package2 also holds a sub-package: subpkg
- Each module defines one function (myfunc1...4).
- Files in folders are imported via absolute OR relative imports.

Absolute Imports

- Specify the full path from the project root folder.

Reach myfunc1

```
from package1.module1 import myfunc1
# or
from package1 import module1
module1.myfunc1()
```

Reach myfunc3

```
from package1.subpkg.module3 import myfunc3
# or
from package1.subpkg import module3
module3.myfunc3()
```

✓ Advantages

- Easy to trace modules
- Simple & straightforward
- Survive moving the project

✗ Disadvantage

- Paths get very long for deeply nested / long-named modules

Relative Imports

- Imported relative to the current file's location.
- Add a leading period (.) before the module name with from.
- Use two periods (..) to go one level up.

Syntax

```
from .module1 import myfunc1  
  
from .subpkg.module3 import myfunc3
```

✓ Advantages

- Easy to work with
- Shorter than absolute imports

✗ Disadvantage

- Harder to trace where the code resides

Summary

- `import` lets you reuse functions/objects written in another file or installed via `pip`.
- A module is the code inside a `.py` file (variables, functions, classes).
- Modules break large code into smaller, reusable files.
- `from ... import` brings in only the names you need; aliases (`as`) shorten them.
- A package is a directory of modules, marked by `__init__.py`.
- Python finds modules via: current dir → built-ins → `sys.path`.
- Absolute imports use the full path from the root; relative imports use `.` / `..` notation.

Thank You

Questions & Discussion

CMPE-103 • Unit VI: Modules & Packages